



Université du Québec
à Chicoutimi

Module d'informatique et de mathématique
Département d'informatique et de mathématiques

8INF950 : Sujet spécial en cybersécurité

Cybersécurité défensive : Mapping automatique de base de
données d'incident et de détection

Date : 14 août 2026
8INF950 : Sujet spécial en cybersécurité - Été 2026

Table des matières

1	Synthèse des lectures	3
1.1	Doc 1 — 99% de faux positifs	3
1.2	Doc 2 — UCO (Unified Cybersecurity Ontology)	3
2	Introduction	4
2.1	Problématique et objectifs	4
2.2	Contexte — VERIS	4
2.3	Contexte — MITRE ATT&CK	5
2.4	Contexte — Le mapping	5
2.5	Contexte — Les 7 capability groups	6
2.6	Contexte — Versions et formats de données	6
3	Méthodologie	6
3.1	La méthode actuelle de mapping	6
3.2	État de l'art — Alarmes SIEM et modèle REACT	7
3.3	État de l'art — Ontologies et unification (UCO)	7
3.4	État de l'art — Approches IA pour l'alignement sémantique	7
3.5	Notre pipeline	7
3.5.1	Architecture du projet	9
4	Solution	9
4.1	Solution RAG	9
4.1.1	Principe	9
4.1.2	Flux par capacité VERIS	10
4.1.3	Deux variantes expérimentées	10
4.1.4	Modules logiciels	10
4.1.5	Technologies et configuration	10
4.1.6	Format de sortie	11
4.2	Solution Fine-tune	11
4.3	Solution PROMPT	11
4.3.1	Principe	11
4.3.2	Périmètre traité	11
4.3.3	Architecture logicielle	11
4.3.4	Modèles Together AI	12
4.3.5	Problèmes rencontrés	12
4.3.6	Correctifs effectués	12
4.4	Synthèse des trois solutions	12
5	Résultats	12
5.1	Configuration expérimentale	12
5.2	Résultats — Solution RAG	12
5.3	Résultats — Solution Fine-tune	13
5.4	Résultats — Solution PROMPT	13
5.4.1	Résultat de la première run	13
5.4.2	Seconde run	13
6	Discussion	14

7 Conclusion 14

Table des figures

1	Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.	8
---	--	---

1 Synthèse des lectures

1.1 Doc 1 — 99% de faux positifs

On distingue trois types de faux positifs (FP) :

- **Vraies alarmes** : ce sont les menaces réelles.
- **Fausse alarmes** : le système se déclenche sans raison légitime.
- **Alarmes bénignes** : alarmes techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore sur un processus humain, grâce aux connaissances et à l'expérience des analystes.

Les auteurs proposent le modèle **REACT** : les alarmes doivent être

- **Reliable** (fiables) ;
- **Explained** (explicables) ;
- **Analytical** (analytiques) ;
- **Contextual** (contextuelles) ;
- **Transferable** (transférables à d'autres environnements).

Ce modèle est pensé pour guider la conception d'outils fondés sur l'apprentissage automatique.

1.2 Doc 2 — UCO (Unified Cybersecurity Ontology)

UCO est une ontologie sémantique visant à unifier et intégrer les données hétérogènes de cybersécurité issues de multiples standards. Son but est d'*améliorer la conscience situationnelle des analystes*.

Caractéristiques principales :

- Extension de l'ontologie IDS développée par le même groupe.
- Intégration des standards les plus utilisés en cybersécurité : STIX, CVE, CCE, CVSS, CAPEC, CybOX et la *Kill Chain*.
- Représentation en OWL/RDF plutôt qu'en XML, ce qui autorise le raisonnement automatique.
- 106 classes, 59 propriétés d'objet et 45 propriétés de données.
- Expressivité logique de niveau *ALCROIQ(D)*.

Apport. Inférence logique, mappée au *Linked Open Data* (DBpedia, YAGO, etc.), permettant le croisement des données cyber avec des connaissances générales.

Cas d'usage. Un prototype identifie les vulnérabilités liées à un type de logiciel à partir des données NVD. Au moyen de requêtes SPARQL, il recense les produits d'un éditeur et leurs vulnérabilités, afin de suggérer une alternative exempte de vulnérabilités ou de comparer l'impact sécuritaire d'un changement de fournisseur.

2 Introduction

2.1 Problématique et objectifs

Dans un centre d'opérations de sécurité (SOC), les analystes doivent constamment relier des incidents décrits dans un vocabulaire normalisé aux techniques adverses documentées dans des référentiels offensifs. Ce travail d'alignement sémantique permet d'enrichir la détection, contextualiser les alertes et améliorer la conscience situationnelle, mais il reste en grande partie manuel, coûteux et peu reproductible.

L'objectif de ce projet est de reproduire automatiquement, à l'aide de l'intelligence artificielle, le mapping entre le vocabulaire VERIS et le framework MITRE ATT&CK, puis de mesurer la qualité des mappings produits par rapport à la référence officielle publiée par le CTID (*Mappings Explorer*).

Nous avons choisi de mettre trois approches en concurrence :

- Un modèle **RAG** (Retrieval-Augmented Generation)
- Un modèle d'ia modifié pour être **fine-tuné** (fine-tuning supervisé)
- Un modèle d'ia généraliste avec **prompt engineering** (ingénierie de prompts).

Le but de notre contribution est de venir créer :

- Un pipeline reproductible permettant de générer les mappings entre VERIS et MITRE ATT&CK (préparation des données, génération, comparaison) ;
- Un format de sortie standardisé pour les mappings générés par nos solutions ;
- Des scripts de comparaison automatisée entre le mapping existant et les mappings générés par nos solutions ;

Le rapport s'organise comme suit :

- L'introduction présente le contexte technique (§2.2 à §2.6) ;
- La méthodologie décrit la pratique actuelle, l'état de l'art et notre cadre expérimental ;
- La section *Solution* détaille les trois approches retenues ;
- Les résultats, la discussion et la conclusion clôturent l'analyse.

2.2 Contexte — VERIS

VERIS (*Vocabulary for Event Recording and Incident Sharing*) est un vocabulaire structuré conçu pour décrire de manière homogène les incidents de cybersécurité. Son objectif principal est de fournir un cadre commun pour documenter ce qui s'est produit lors d'un incident, indépendamment de l'organisation ou de l'outil utilisé. Autrement dit, VERIS sert à normaliser la description des événements de sécurité afin de faciliter leur analyse, leur comparaison et leur partage entre analystes, équipes de réponse à incident et organismes de recherche.

Sur le plan de son organisation, VERIS repose sur une structure hiérarchique. Le vocabulaire est découpé en grands axes, comme **action**, **attribute**, **actor** ou encore **value_chain**, qui correspondent à différentes dimensions d'un incident. Chaque axe contient ensuite des catégories plus précises, par exemple **hacking**, **malware** ou **social** dans l'axe **action**, puis des sous-catégories et des valeurs plus fines telles que **variety** ou **vector**. Cette organisation permet de représenter un incident avec plusieurs niveaux de granularité, depuis une vue générale jusqu'à une description beaucoup plus détaillée du comportement observé.

VERIS a été développé à l'origine dans le cadre des travaux de Verizon autour du *Data Breach Investigations Report* (DBIR), avec l'idée de disposer d'un schéma cohérent pour collecter et comparer les données d'incidents à grande échelle. Avec le temps, il est devenu un standard largement reconnu dans le domaine de la réponse à incident et de l'analyse cyber, notamment parce qu'il permet d'agréger des observations provenant de sources variées tout en conservant une structure commune. Dans le cadre de ce projet, VERIS joue donc le rôle de référentiel décrivant le “quoi” de l'incident, que l'on cherche ensuite à relier au “comment” représenté par les techniques de MITRE ATT&CK.

2.3 Contexte — MITRE ATT&CK

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) est une base de connaissances dédiée à la description des comportements adverses observés lors d'intrusions réelles. Contrairement à un simple catalogue de vulnérabilités ou d'outils, ATT&CK cherche à modéliser la manière dont un attaquant agit au cours d'une opération, c'est-à-dire les étapes, les objectifs intermédiaires et les techniques mobilisées. Dans cette perspective, ATT&CK décrit le “comment” d'une attaque, là où d'autres référentiels, comme VERIS, décrivent davantage le “quoi” de l'incident.

Le framework est organisé de manière hiérarchique autour de **tactiques** et de **techniques**. Les tactiques représentent les grands objectifs poursuivis par l'attaquant, par exemple l'accès initial, l'exécution, la persistance, l'élévation de privilèges ou l'impact. À l'intérieur de chaque tactique, ATT&CK recense des techniques identifiées par un code de type T1059, accompagnées d'un nom, d'une description et d'un ensemble de tactiques associées. Certaines techniques sont elles-mêmes décomposées en **sous-techniques**, notées par exemple T1059.001, ce qui permet de représenter plus finement des variantes de comportement ou des modes opératoires particuliers.

MITRE ATT&CK est maintenu par la MITRE Corporation à partir de retours d'expérience opérationnels, de rapports d'incidents et d'observations issues du renseignement sur les menaces. Il s'est progressivement imposé comme une référence majeure en cybersécurité défensive, notamment pour la détection, la chasse aux menaces, l'évaluation de couverture et la structuration des connaissances sur les attaques. Dans le cadre de ce projet, ATT&CK constitue le référentiel cible vers lequel les capacités VERIS sont mappées, afin de relier la description d'un incident aux techniques adverses susceptibles d'en expliquer le déroulement.

2.4 Contexte — Le mapping

Le mapping étudié dans ce projet consiste à relier des capacités issues du vocabulaire VERIS, qui décrivent la nature et les caractéristiques d'un incident, à des techniques du framework MITRE ATT&CK, qui décrivent les comportements et modes opératoires adverses susceptibles d'expliquer cet incident. Autrement dit, il s'agit de faire le lien entre le “quoi” observé dans VERIS et le “comment” modélisé dans ATT&CK. Concrètement, une capacité VERIS, par exemple `action.hacking.variety.SQLi`, peut être associée à une ou plusieurs techniques ATT&CK jugées pertinentes par les experts.

Ces correspondances sont produites et publiées par le CTID / MITRE Engenuity. Elles constituent dans notre travail une double référence : d'une part, elles définissent la cible que nos solutions d'intelligence artificielle cherchent à reproduire automatiquement ; d'autre part, elles servent de base d'évaluation pour mesurer la qualité des mappings

générés. Le problème traité dans ce projet n'est donc pas seulement de produire des associations plausibles entre VERIS et ATT&CK, mais bien de se rapprocher le plus possible d'un mapping expert déjà établi.

2.5 Contexte — Les 7 capability groups

Sept groupes VERIS dits « comportementaux » sont mappés vers ATT&CK. Le Tableau 1 présente les volumes pour la paire de référence VERIS 1.4.1 / ATT&CK 19.1.

Capability group	Paires VERIS↔ATT&CK	Capacités VERIS
action.hacking	538	68
action.malware	405	54
action.social	78	23
attribute.integrity	91	13
attribute.confidentiality	73	1
attribute.availability	44	6
value_chain.development	25	12

TABLEAU 1 – Les sept capability groups VERIS mappés (référence 1.4.1 / 19.1).

Cas particulier : pour `attribute.confidentiality`, les experts mappent le champ `data_disclosure` lui-même (une seule capacité), et non des valeurs sous `variety`.

2.6 Contexte — Versions et formats de données

Quatre paires de versions sont supportées de bout en bout : ATT&CK 9.0 / VERIS 1.3.5, 12.1 / 1.3.7, 16.1 / 1.4.0 et 19.1 / 1.4.1.

La **version cible d'évaluation** de ce rapport est VERIS 1.4.1 / ATT&CK 19.1 (clé `veris-1.4.1_attack-19.1-enterprise`).

Les entrées préparées pour chaque solution sont :

- `veris_<v>.json` : capacités VERIS dans les sept groupes (les « questions ») ;
- `attack_<a>.json` : techniques ATT&CK non dépréciées (les « candidats »).

Les sorties des solutions suivent le format `veris_to_mitre` ; la référence experte utilise le format CTID (`mapping_objects`).

3 Méthodologie

3.1 La méthode actuelle de mapping

Aujourd'hui, le mapping VERIS → ATT&CK est réalisé par des experts du CTID / MITRE Engenuity [?]. Le processus est essentiellement **manuel** : pour chaque capacité VERIS, l'expert analyse sa sémantique, consulte le catalogue ATT&CK et sélectionne les techniques correspondantes (y compris les sous-techniques lorsque nécessaire). Les résultats sont publiés via l'outil *Mappings Explorer* au format CTID.

Cette approche présente des **avantages** indiscutables : haute qualité, validation par des spécialistes et référence largement adoptée. Ses **limites** sont toutefois significatives : coût en temps humain, faible scalabilité et nécessité de recommencer le travail à chaque

nouvelle version de VERIS ou d'ATT&CK. Notre projet vise précisément à automatiser cette tâche tout en mesurant l'écart par rapport à cette référence experte.

3.2 État de l'art — Alarmes SIEM et modèle REACT

On distingue trois types de faux positifs (FP) dans les systèmes SIEM [?] :

- **Vraies alarmes** : menaces réelles.
- **Fausse alarmes** : déclenchement sans raison légitime.
- **Alarmes bénignes** : techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore largement sur l'expertise humaine. Les auteurs proposent le modèle **REACT** : les alarmes doivent être **R**eliable (fiables), **E**xplained (explicables), **A**nalytical (analytiques), **C**ontextual (contextuelles) et **T**ransférable (transférables). Ce cadre guide la conception d'outils fondés sur l'apprentissage automatique. Dans notre projet, l'automatisation du mapping VERIS↔ATT&CK vise à réduire la charge cognitive des analystes en normalisant plus rapidement le lien entre incidents et techniques adverses.

3.3 État de l'art — Ontologies et unification (UCO)

UCO (*Unified Cybersecurity Ontology*) [?] est une ontologie sémantique visant à unifier des données hétérogènes de cybersécurité (STIX, CVE, CAPEC, CybOX, *Kill Chain*, etc.) en OWL/RDF, ce qui autorise le raisonnement automatique. L'alignement VERIS ↔ ATT&CK s'inscrit dans cette problématique plus large d'**unification de taxonomies cyber** : deux vocabulaires distincts doivent être reliés de manière cohérente pour améliorer la conscience situationnelle des analystes.

3.4 État de l'art — Approches IA pour l'alignement sémantique

Trois familles d'approches sont comparées dans ce projet :

- **Prompt engineering** : un LLM généraliste produit le mapping à partir d'instructions (zero-shot ou few-shot).
- **Fine-tuning** : un modèle est entraîné sur des paires expert annotées. Spécialisation forte et nous supposons une faible hallucination, mais avec un coût d'entraînement plus élevé.
- **RAG** [?] : un contexte pertinent (techniques ATT&CK, exemples experts) est récupéré avant la décision. Compromis entre interprétabilité, coût et performance.

3.5 Notre pipeline

Notre pipeline expérimental s'organise en quatre étapes principales, illustrées à la Figure 1. L'objectif est de partir de sources officielles, de produire des mappings automatisés VERIS → ATT&CK à l'aide de trois approches IA, puis de les comparer au mapping expert CTID.

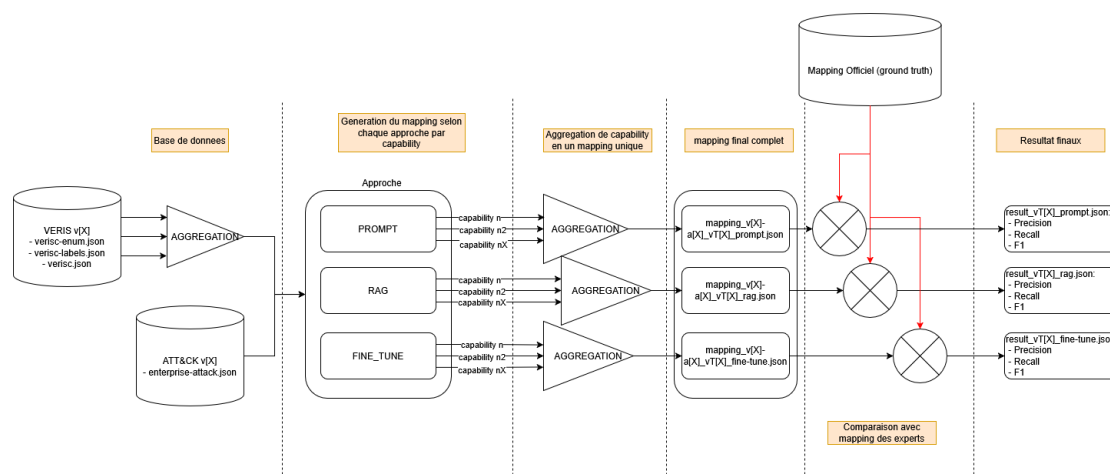


FIGURE 1 – Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.

1. **Téléchargement des données brutes.** Les sources officielles sont rassemblées dans le dossier `data/raw/`. On y trouve les fichiers VERIS (`verisc-enum.json`, `verisc-labels.json`, `verisc.json`), le catalogue ATT&CK au format STIX (`enterprise-attack.json`) ainsi que les mappings experts publiés par le CTID (CSV, YAML, JSON ou STIX selon la version). Quatre paires de versions sont disponibles, de VERIS 1.3.5 / ATT&CK 9.0 à VERIS 1.4.1 / ATT&CK 19.1.
2. **Préparation des entrées.** Le script `tools/build_work_inputs.py` transforme ces données brutes en fichiers exploitables par les solutions IA, stockés dans `data_for_work/`. Pour chaque paire de versions, il produit :
 - `veris_<v>.json` : les capacités VERIS des sept capability groups (les « questions » à mapper) ;
 - `attack_<a>.json` : les techniques ATT&CK non dépréciées (les « candidats ») ;
 - `mapping_des_experts.json` : une copie locale du mapping de référence, utilisée pour la comparaison.
3. **Génération des mappings.** Chacune des trois solutions (`Solution_RAG`, `Solution_PROMPT`, `Solution_FINE_TUNE`) ingère les fichiers préparés et produit **sept fichiers JSON** au format `veris_to_mitre`, soit un fichier par capability group (`action.hacking`, `action.malware`, `action.social`, `attribute.integrity`, `attribute.confidentiality`, `attribute.availability`, `value_chain.development`). Les résultats sont écrits dans `Resultat/Resultat_<APPROCHE>/`.
4. **Comparaison aux experts.** Le script `Resultat/compare_veris_mappings_v2.py` confronte automatiquement les sorties des solutions au mapping expert CTID (`Resultat/Mapping_des_experts/`). Il valide la présence des sept fichiers, calcule les métriques de recouvrement (précision, rappel, F1, Jaccard) sur les paires (`veris_id`, `attack_id`) et classe les solutions entre elles.

Principe anti-fuite. Pour la version cible d'évaluation (VERIS 1.4.1 / ATT&CK 19.1), le mapping expert correspondant n'est **jamais** utilisé comme exemple d'entraînement ou d'indexation dans les solutions IA. Seules les versions antérieures (9.0, 12.1, 16.1) peuvent servir d'exemples analogiques, notamment dans la variante RAG `with_examples`.

3.5.1 Architecture du projet

Le dépôt SIEM est organisé de manière modulaire afin de séparer clairement les données, leur préparation, l'implémentation des solutions IA et l'évaluation des résultats. Cette organisation facilite la reproductibilité du pipeline et permet de traiter plusieurs paires de versions VERIS / ATT&CK dans un même cadre expérimental.

- **data/** : contient les **sources brutes** issues des dépôts officiels. Le sous-dossier **raw/attack/** regroupe les catalogues ATT&CK au format STIX, **raw/veris/** les schémas et énumérations VERIS, et **raw/mapping/** les mappings experts publiés par le CTID. Le dossier **databases/** regroupe également des bases SQLite auxiliaires utilisées pour certains vocabulaires et correspondances.
- **data_for_work/** : regroupe les **entrées préparées** pour chaque paire de versions, dans des sous-dossiers nommés selon la convention **attack-<a>_veris-<v>**. Chaque dossier contient les fichiers **veris-<v>.json** (capacités VERIS à mapper), **attack-<a>.json** (techniques ATT&CK candidates) et **mapping_des_experts.json** (copie locale de la référence experte).
- **tools/** : regroupe les **scripts de préparation et de validation** du pipeline. Le script **build_work_inputs.py** transforme les données brutes en fichiers exploitables dans **data_for_work/**, tandis que **build_example_results.py** génère des sorties de référence au format **veris_to_mitre**, utiles pour valider le format attendu.
- **Solution/** : contient le **code des trois approches IA** mises en concurrence : **Solution_RAG**, **Solution_PROMPT** et **Solution_FINE_TUNE**. Chaque solution lit les mêmes entrées préparées et produit sept fichiers JSON, un par capability group VERIS.
- **Resultat/** : centralise les **sorties expérimentales** et les outils de comparaison. On y trouve les mappings experts officiels dans **Mapping_des_experts/**, les résultats produits par chaque approche dans **Resultat_RAG/**, **Resultat_PROMPT/** et **Resultat_FINE_TUNE/**, ainsi que les scripts **compare_veris_mappings.py** et **compare_veris_mappings_v2.py** utilisés pour mesurer la qualité des mappings générés.

Cette architecture matérialise le flux décrit précédemment : les données brutes sont d'abord normalisées, puis transformées en entrées de travail, ensuite traitées par les solutions IA, et enfin comparées au mapping expert CTID.

4 Solution

Les trois solutions partagent les mêmes entrées (**veris-<v>.json**, **attack-<a>.json**) et la même sortie attendue (sept fichiers **veris_to_mitre**), mais diffèrent par leur stratégie de décision IA.

4.1 Solution RAG

4.1.1 Principe

La solution **RAG** ancre chaque décision dans du contexte récupéré via deux corpus vectoriels ChromaDB :

1. **attack_techniques** — catalogue ATT&CK cible (les candidats);

2. **expert_examples** — mappings experts des versions antérieures (exemples analogues).

Pour chaque capacité VERIS, le système récupère les techniques les plus similaires (top- k) et, le cas échéant, des exemples de mappings experts (top- m), puis produit une décision structurée en JSON.

4.1.2 Flux par capacité VERIS

Le traitement suit la chaîne suivante : (1) récupération des candidats ATT&CK ; (2) récupération des exemples experts (variante **with_examples**) ; (3) décision via le backend configuré (**retrieval**, **local_llm** ou **llm Azure**) ; (4) validation anti-hallucination (rejet des identifiants hors catalogue) ; (5) enrichissement des libellés et tactiques depuis le catalogue local ; (6) agrégation en entrées **veris_to_mitre** regroupées par capability group.

4.1.3 Deux variantes expérimentées

Mode	Corpus de récupération	Dossier de sortie
attack_only	ATT&CK seul	<code>..._RAG_attack_only/</code>
with_examples	ATT&CK + exemples experts	<code>..._RAG_with_examples/</code>

TABLEAU 2 – Variantes expérimentées de la solution RAG.

4.1.4 Modules logiciels

Fichier	Rôle
<code>config.py</code>	Versions, chemins, hyperparamètres
<code>ingest.py</code>	Indexation ChromaDB
<code>retrieve.py</code>	Récupération top- k / top- m
<code>prompt.py</code>	Prompts et schéma JSON
<code>generator.py</code>	Backends de génération
<code>generate_mapping.py</code>	Pipeline complet → 7 fichiers

TABLEAU 3 – Modules principaux de la solution RAG (**veris_mapping/**).

4.1.5 Technologies et configuration

En mode local (défaut), les embeddings sont produits par **sentence-transformers** et la génération peut s'appuyer sur la seule similarité cosinus (**retrieval**), sans LLM. En mode Azure (optionnel), les embeddings et la génération passent par Azure OpenAI. Les paramètres clés sont **RAG_TOP_K_TECHNIQUES** (20 par défaut) et **RAG_TOP_M_EXAMPLES** (5 par défaut).

4.1.6 Format de sortie

Chaque fichier de résultat contient une liste `veris_to_mitre` avec, pour chaque capacité, les techniques proposées, un score de confiance et une justification. Les identifiants ATT&CK invalides sont rejetés ; noms et tactiques proviennent du catalogue local.

4.2 Solution Fine-tune

4.3 Solution PROMPT

4.3.1 Principe

La solution **PROMPT** (dossier `Solution/Solution_PROMPT/`) s'appuie sur un LLM généraliste interrogé via l'API **Together AI**, sans index vectoriel ni fine-tuning. L'approche est purement fondée sur le *prompt engineering* en mode **zero-shot** : un *system prompt* fixe le rôle, les règles anti-hallucination et le schéma JSON attendu, un *user prompt* fournit, pour chaque capability group VERIS, les données de référence et l'instruction de mapping.

Contrairement à la solution RAG, aucun contexte n'est récupéré dynamiquement : le modèle reçoit les bases de données préparées (`veris_<v>.json` et `attack_<a>.json`) — filtrées afin d'alléger le nombre de tokens en entrée — et doit produire des **liens d'identifiants** (`veris_id` → liste d'`attack_ids`). Les libellés, les tactiques et le sens MITRE → VERIS sont ensuite reconstruits localement à partir des mêmes catalogues, sur le même principe que l'enrichissement post-génération du RAG, via le script `reconstruct_mapping.py`.

4.3.2 Périmètre traité

Le traitement est découpé selon les sept capability groups du mapping expert :

- `action.hacking`, `action.malware`, `action.social` ;
- `attribute.integrity`, `attribute.confidentiality`, `attribute.availability` ;
- `value_chain.development`.

Une requête LLM est émise **par capability group**. Les résultats finaux sont écrits dans `Resultat/Resultat_PROMPT/veris-<v>_attack-<a>-enterprise_PROMPT_<modèle>/`, sous la forme de fichiers `<capability>.json`.

4.3.3 Architecture logicielle

Le pipeline combine `run_mapping.py` (génération) et `reconstruct_mapping.py` (enrichissement) :

1. chargement des entrées (`-dw` ou couple `-a/-v`) (par défaut, les dernières versions des bases de données) ;
2. filtrage VERIS au groupe courant et compactage ATT&CK (descriptions tronquées à 250 caractères) ;
3. construction d'un user prompt par capability et chargement de `system-prompt.md` ;
4. génération LLM séquentielle (timeout 900 s, `max_tokens=100000`) ;
5. écriture des JSON bruts (IDs) dans `_raw/` ;
6. reconstruction locale du mapping complet (enrichissement + inversion `mitre_to_veris`) directement dans les `*.json` du dossier de résultats, au format attendu par les scripts de comparaison.

Élément	Rôle
<code>run_mapping.py</code>	Orchestration, prompts, appels API, écriture
<code>reconstruct_mapping.py</code>	Enrichissement local IDs, production JSON finaux
<code>system-prompt.md</code>	Rôle CTI, règles, schéma JSON de sortie
<code>requirements.txt</code>	Dépendance <code>together</code>

TABLEAU 4 – Composants de la solution PROMPT

4.3.4 Modèles Together AI

Deux modèles serverless ont été retenus : `moonshotai/Kimi-K3` et `deepseek-ai/DeepSeek-V4-Pro`. Ils ont été choisis parce qu'ils sont accessibles en serverless avec la clé API du projet (contrairement à Qwen 235B et DeepSeek-R1-0528, qui exigent un endpoint dédié), qu'ils offrent une grande fenêtre de contexte adaptée à nos prompts volumineux, et qu'ils représentent deux profils complémentaires : Kimi plus sélectif, DeepSeek plus couvrant en termes de rappel.

4.3.5 Problèmes rencontrés

Initialement, les catalogues étaient envoyés en entier aux modèles et la consigne demandait un mapping bidirectionnel en réponse. Les premières exécutions se sont donc conclues par des timeouts, des fichiers vides et des sorties tronquées.

4.3.6 Correctifs effectués

Les principaux correctifs effectués afin de résoudre ces problèmes ont été :

- réduction des données d'entrée (filtre VERIS + ATT&CK compact) ;
- réduction de la sortie LLM aux seuls IDs ;
- reconstruction post-génération du mapping complet pour la comparaison avec le mapping expert ;
- traitement séquentiel, validation JSON, conservation des bruts dans `_raw/`.

4.4 Synthèse des trois solutions

5 Résultats

5.1 Configuration expérimentale

5.2 Résultats — Solution RAG

Le Tableau 5 résume les performances globales des deux variantes RAG. L'injection d'exemples experts des versions antérieures **double le F1** (27,4 % contre 9,8 %), principalement grâce à un gain de rappel.

Variante	Précision	Rappel	F1
RAG_with_examples	18,9 %	49,7 %	27,4 %
RAG_attack_only	7,2 %	15,4 %	9,8 %

TABLEAU 5 – Résultats globaux RAG vs mapping expert (mode `retrieval` local).

Le mode `retrieval` privilégie le rappel au détriment de la précision : de nombreuses paires expertes sont retrouvées, mais beaucoup de propositions sont incorrectes. Certaines familles de sous-techniques (par exemple T1546.*) restent partiellement couvertes lorsque le *top-k* est insuffisant.

5.3 Résultats — Solution Fine-tune

5.4 Résultats — Solution PROMPT

5.4.1 Résultat de la première run

L'expérimentation du mapping se base sur les versions suivantes : VERIS 1.4.1 et ATT&CK 19.1 via Together AI. Le Tableau 6 résume le **premier run** (pipeline initial, avant correctifs).

Capability	Kimi-K3	DeepSeek-V4-Pro
action.hacking	JSON tronqué	vide
action.malware	JSON tronqué	vide
action.social	JSON tronqué	vide
attribute.availability	complet	vide
attribute.confidentiality	vide	vide
attribute.integrity	vide	vide
value_chain.development	vide	vide

TABLEAU 6 – Premier run PROMPT (avant correctifs de pipeline).

Kimi-K3. Seule la capability `attribute.availability` (petit groupe) a produit un JSON valide ; les autres (plus larges) ont été coupées en sortie.

DeepSeek-V4-Pro. Les sept fichiers sont restés vides (timeout et/ou tokens de raisonnement sans `content` exploitable).

5.4.2 Seconde run

Après les correctifs appliqués (voir 4.3.6), une nouvelle expérimentation a été effectuée avec les mêmes versions des bases de données (VERIS 1.4.1 et ATT&CK 19.1). Cette fois, les sept fichiers JSON de capability sont complets et valides pour les deux modèles. Ils sont ensuite enrichis au format attendu par le script de comparaison (`compare_veris_mappings_v2.py`). Le Tableau 7 compare les performances globales agrégées au mapping expert.

Modèle	Paires sol.	Communes	Préc.	Rapp.	F1
Kimi-K3	770	298	38,7 %	23,8 %	29,5 %
DeepSeek-V4-Pro	2231	523	23,4 %	41,8 %	30,0 %

TABLEAU 7 – Second run PROMPT vs mapping expert (agrégé, 1250 paires expertes).

Capability	Exp.	Kimi-K3			DeepSeek	
		Prec.	Rapp.	F1	Prec.	F1
action.hacking	534	37,3	25,1	30,0	21,3	25,1
action.malware	405	40,0	17,3	24,1	21,1	28,6
action.social	79	31,7	25,3	28,2	31,2	31,4
attribute.integrity	91	25,3	26,4	25,8	13,4	21,3
attribute.confidentiality	73	100,0	26,0	41,3	59,0	72,6
attribute.availability	44	58,8	45,5	51,3	52,7	58,6
value_chain.development	25	44,0	44,0	44,0	88,9	47,1

TABLEAU 8 – Second run PROMPT : détail par capability group (%).

Les deux modèles atteignent un F1 global de ≈ 30 %, voisin du meilleur RAG local (`with_examples`, 27,4 %). Ils divergent surtout sur le compromis. En effet, on a deux profils :

- Kimi-K3 propose moins de paires avec une précision plus haute ($\approx 38,7$ %);
- DeepSeek-V4-Pro, quant à lui, propose beaucoup plus de paires avec un rappel supérieur ($\approx 41,8$ %).

On remarque donc que **Kimi-K3** fait un mapping plus sélectif, tandis que **DeepSeek-V4-Pro** fait une couverture plus large, au prix de davantage de faux positifs.

Le gain le plus marqué apparaît sur `attribute.confidentiality` (DeepSeek F1 72,6 %) et `attribute.availability` (F1 > 50 % pour les deux), alors que les grands groupes `hacking/malware` restent les plus difficiles.

6 Discussion

7 Conclusion